FRENCH UNIVERSITY IN ARMENIA

Faculty of Computer Science and Applied Mathematics

Final Report

FLOCK - Emergency Evacuation Management Database

Subject: Database 1

Prepared by: Lusine STEPANYAN

Elen YEGHIAZARYAN

Melanya MARTIROSYAN

Supervised by: Varazdat AVETISYAN, PhD

April 6, 2026

Contents

1. Conceptual Design 2
1.1 Problem Domain Analysis 2
1.2 System Requirements 3
1.3 Main Data Objects 5
1.4 Business Rules 7
1.5 System Data Requirements 9
1.6 System Data Processes 11
1.7 UML Diagrams 13
2. Logical Design 17
2.1 Entity-Relationship (ER) Model 18
2.2 Entities, Attributes, and Relationships 20
2.3 Relationship Cardinalities and Constraints 29
2.4 Relational Scheme 33
2.5 Tables, Primary Keys & Foreign Keys 34
2.6 Normalization Process 38
2.7 Relational Algebra Operations 42
3. Physical Implementation 53
3.1 DDL - Database and Table Creation 54
3.2 DML - Data Insertion 56
3.3 DQL - Data Retrieval Queries 58
3.4 Views 63
3.5 Indexes 65
3.6 Triggers 65
3.7 Stored Procedures 65
3.8 DCL - Access Control 65
3.9 Database Deployment Description 65
3.10 Project Conclusions and Results 65

1. Conceptual Design

The conceptual design phase establishes the foundation of the Smart Evacuation Navigation System by identifying what the system must do, who uses it, and what information it must manage. This section covers the problem domain analysis, system requirements, main data objects, business rules, data requirements, data processes, and supporting UML diagrams.

1.1 Problem Domain Analysis

The Smart Evacuation Navigation System is a B2B safety platform for large multi-story buildings. During an emergency, occupants need immediate, reliable guidance to the nearest safe exit. The system bridges the gap between static architectural floor plans and real-time dynamic navigation by converting building layouts into a graph structure and running pathfinding algorithms on it.

Target Environments

The system is designed for deployment in the following types of buildings:

- Shopping malls
- Office towers
- Hospitals
- Universities
- Airports
- Stadiums

Core problem: In a fire or emergency, people panic and may not know the nearest exit, especially in large unfamiliar buildings. Existing signage is static and cannot account for blocked exits, active fire zones, or an individual occupant's actual current location. A dynamic, real-time system is required to compute and deliver personalised evacuation routes.

The system solves this by combining Bluetooth Low Energy (BLE) beacon positioning with a graph-based building model. When an emergency is registered, the system queries which exits are still available, and a Breadth-First Search (BFS) algorithm computes the shortest safe path from each occupant's detected position to the nearest open exit. The resulting step-by-step route is delivered directly to the occupant's smartphone.

1.2 System Requirements

The system requirements are divided into functional requirements (what the system must do) and non-functional requirements (the quality constraints it must satisfy).

Functional Requirements

These requirements define the specific tasks and behaviors the system must support for its different users:

User Management and Roles

- The system must allow users to create accounts and log in using their email and password.
- The system must support role-based access (admin and occupant).
- Admins must be able to manage user accounts.

Building and Structure Management

- Businesses must be able to register buildings with name, address, type, and total number of floors.
- Admins must be able to add floors, zones, rooms, and exits to a building.
- The system must store the building layout as a graph where nodes represent locations and edges represent paths.

Beacon and Location Detection

- Admins must be able to assign BLE beacons to floors and zones.
- The system must detect the user's current location using the nearest beacon signal.
- The system must log user location with timestamps.

Emergency and Incident Management

- Admins must be able to create, update, and resolve incidents.
- The system must store incident type, severity level, and status.
- The system must identify active incidents and unsafe areas (blocked exits, fire zones, etc.).

Evacuation and Navigation

- The system must calculate the shortest safe evacuation route using a BFS algorithm.
- The system must avoid unsafe or blocked paths.
- The system must guide users with step-by-step navigation instructions.

History and Data Tracking

- The system must store all evacuation sessions, routes, and route steps.
- Admins must be able to view evacuation history filtered by building, incident, or user.

Non-Functional Requirements

These requirements define performance and system constraints.

Requirement	Description
Performance	Route calculation must respond within 2 seconds under normal operating conditions.
Availability	The system must remain operational during partial network degradation by caching the building graph locally on the device.
Data Integrity	All incident and route history data must be stored with no loss - deletions are not permitted on core log tables.
Access Control	Admin privileges must be strictly separated from occupant access, enforced at the database level through roles and permissions.
Scalability	The database must support many concurrent users in the same building simultaneously during a mass evacuation event.
Uniqueness	Beacon UUIDs must be globally unique across the entire system to prevent incorrect floor detection.
Portability	The system must be deployable for any large building type including shopping malls, hospitals, office towers, airports, and universities.

1.3 Main Data Objects

The conceptual design of the evacuation system identifies several **Main Data Objects** (entities) that are central to the system's operation. These objects store the information required to manage buildings, track users, handle emergencies, and compute safe evacuation routes.

User and Account Objects

- **Users**: Store personal information, contact details, authentication credentials, and roles (either admin or occupant).
- **User Locations**: Record real-time positions of users by linking each detection event to a beacon, floor, and optionally a zone, with a timestamp.

Building and Infrastructure Objects

- **Buildings**: Represent registered buildings with attributes like name, address, type, and total number of floors.
- **Floors**: Each floor belongs to a building and includes the floor number, optional map reference, and height in meters.
- **Zones**: Areas within floors (rooms, corridors, halls), optionally linked to beacons or exits.
- **Exits**: Points of egress, including emergency exits, with attributes such as type, status (available/blocked), and emergency designation.
- **Staircases**: Vertical connections between floors, which can serve multiple floors.

Beacon and Navigation Objects

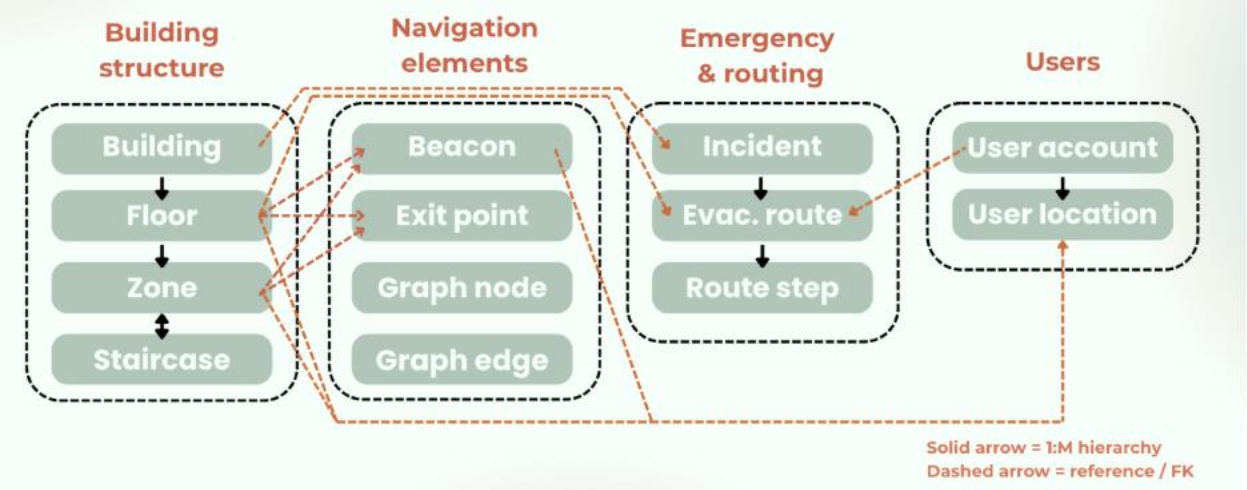
- **Beacons**: Bluetooth Low Energy (BLE) devices installed on floors or zones to detect user presence. Each beacon stores a unique UUID, major/minor values, installation date, and status.
- **Nodes**: Represent points in the building graph (rooms, intersections, exits) for navigation.
- **Edges**: Define walkable paths between nodes, including distance information.

Incident and Evacuation Objects

- **Incidents**: Represent emergencies or abnormal events affecting a building. Attributes include type, severity, status, start/end times, and description.
- **Evacuation Routes**: Computed paths for users during incidents, linked to the user, starting floor, and destination exit.
- **Route Steps**: Ordered instructions along an evacuation route, linked to floors and optional zones, with text guidance.

Figure 1

Main Data Objects



1.4 Business Rules

Business rules define the constraints and policies that govern how data is created, modified, and related within the system. These rules are enforced at both the application and database levels.

Building and Floor Rules

- Each **floor** belongs to exactly one **building**.
- Floor numbers must be **unique within a building** (no two floors in the same building can share the same number).
- A **staircase** can span multiple floors, creating a many-to-many relationship via the StaircaseFloor table.

Zone, Beacon, and Navigation Rules

- Each **zone** belongs to exactly one floor.
- Each **beacon** belongs to exactly one floor and optionally one zone.
- **Beacon UUIDs** must be globally unique across the entire system.
- A new UserLocation record must reference a beacon and a floor that are properly linked in the database.

User and Location Rules

- Each **user** can have many UserLocation records over time.
- Each location record captures **one detected position at a specific moment**.
- Only users with the **admin role** can register incidents or modify the building structure.

Incident and Evacuation Rules

- Each **incident** occurs in one building but can affect many floors.
- Each **evacuation route** must be associated with exactly one incident and one user.
- Each route_id must be **globally unique**.
- An **evacuation route** consists of ordered RouteStep records with floor and optional zone references.
- When an incident is resolved, its end_time must be **after** its start_time.

Exit and Safety Rules

- Each **exit point** belongs to exactly one floor.
- During an active incident, at least one exit per floor must remain **available** at all times.

1.5 System Data Requirements

This section describes the data the system must store, its granularity, and associated constraints.

Building & Physical Structure Data

The system must store information for each registered building, including name, address, type, and total number of floors. Each building contains multiple floors, and each floor must be uniquely identifiable by its floor number within the building. Floors are subdivided into zones, each with a name and type (hallway, room, stairwell, lobby, etc.). The system must also store staircase data, where a single staircase can span multiple floors and is linked to each floor it serves through a junction table.

Beacon Data

For each beacon deployed in a building, the system must store its UUID, major value, minor value, installation date, status (active or inactive), and the floor - and optionally the zone - it is assigned to. Beacon UUIDs must be globally unique across all buildings.

Exit & Safety Data

Each exit point must be stored with its name, type (door, stairwell exit, fire escape, etc.), emergency designation (true/false), and current status (available or blocked). Each exit belongs to exactly one floor and optionally to one zone.

User & Account Data

The system must store user accounts with full name, phone number, email (unique), role (admin or occupant), and company name. Roles determine the actions a user can perform in the system.

Location Detection Data

Every time a user's device detects a beacon, the system must create a location record capturing the user, the detected beacon, the resolved floor, the resolved zone (if any), and the exact timestamp of detection. This data forms the audit trail of user movement during an incident.

Incident Data

Each incident must be stored with the building it occurred in, its type (fire, gas leak, flood, etc.), severity level (low, medium, high, critical), start time, end time (nullable — set when resolved), current status (active or resolved), and a free-text description. The end time must always be after the start time when set.

Evacuation Route Data

When a route is generated for a user during an incident, the system must store the route record linking the incident, the user, starting floor, and destination exit. Each route also has a status (generated, completed, abandoned) and a creation timestamp.

Route Step Data

Each evacuation route is broken into ordered steps. Every step must store its sequence number (unique within the route), the floor it occurs on, optional zone, and a plain-text instruction (e.g., "Turn left toward Zone B, proceed to staircase 2"). This is the data delivered to the user's application for guidance.

1.6 System Data Processes

The following processes define the key operations the system must support, from initial setup through real-time evacuation and post-incident review.

ID	Process Name	Description
P1	**Building Registration**	An admin registers a building, then adds floors, zones, exits, and staircases. Beacons are assigned to floors and optionally to zones.
P2	**Graph Construction**	The system constructs a navigable graph from the building layout where rooms, intersections, and exits are nodes and walkable corridors are edges. DFS traversal validates full graph connectivity.
P3	**User Floor Detection**	When a user opens the app, their device scans for nearby BLE beacons. The closest beacon ID is sent to the system, which looks up the corresponding floor and zone. A UserLocation record is created with a timestamp.
P4	**Incident Registration**	An admin registers an incident (fire, gas leak, etc.) for a building, setting its type, severity, and affected floors. Exits on affected floors are flagged as unavailable.
P5	**Route Generation**	The system takes the user's current floor (from P3), the active incident (from P4), and the list of available exits. BFS is run on the building graph to find the shortest safe path to the nearest open exit. The resulting route and its steps are saved to the database.
P6	**Route Delivery**	The computed route steps are returned to the user's smartphone as an ordered, step-by-step guidance list, similar to turn-by-turn navigation.

P7	**Session Logging & Review**	Every evacuation session — who was evacuated, from which floor, via which route, during which incident — is permanently logged and available for post-incident review by authorised admins.

1.7 UML Diagrams

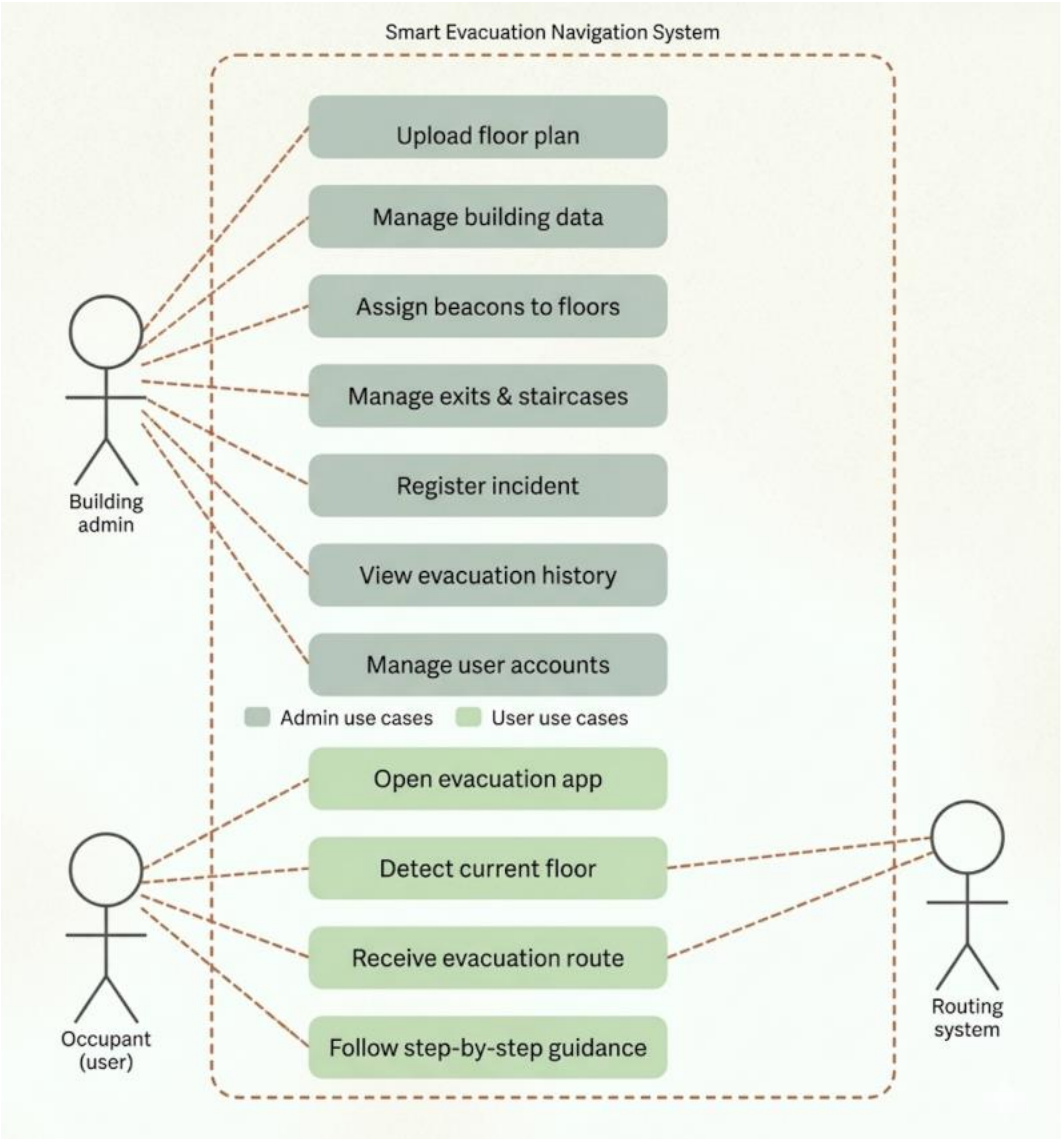
1. Use Case Diagram

The use-case diagram identifies all actors in the system and the operations each actor can perform.

Three actors are defined: the Building Admin, the Occupant and the Routing System.

Figure 2

Use Case Diagram

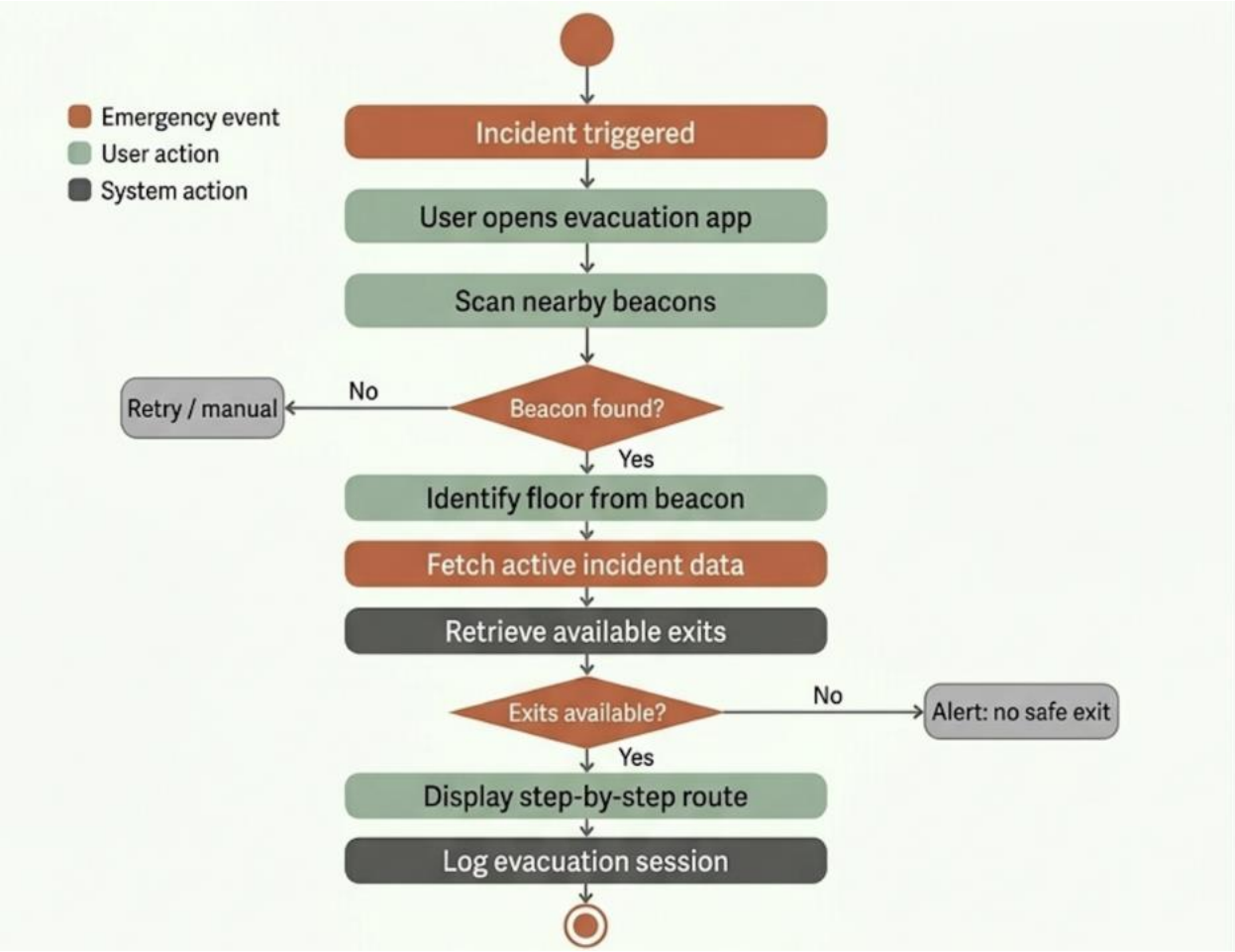


2. Activity Diagram

The activity diagram captures the complete evacuation flow from the moment an emergency begins to the moment an occupant exits the building. It includes two critical decision points with defined fallback paths.

Figure 3

Activity Diagram

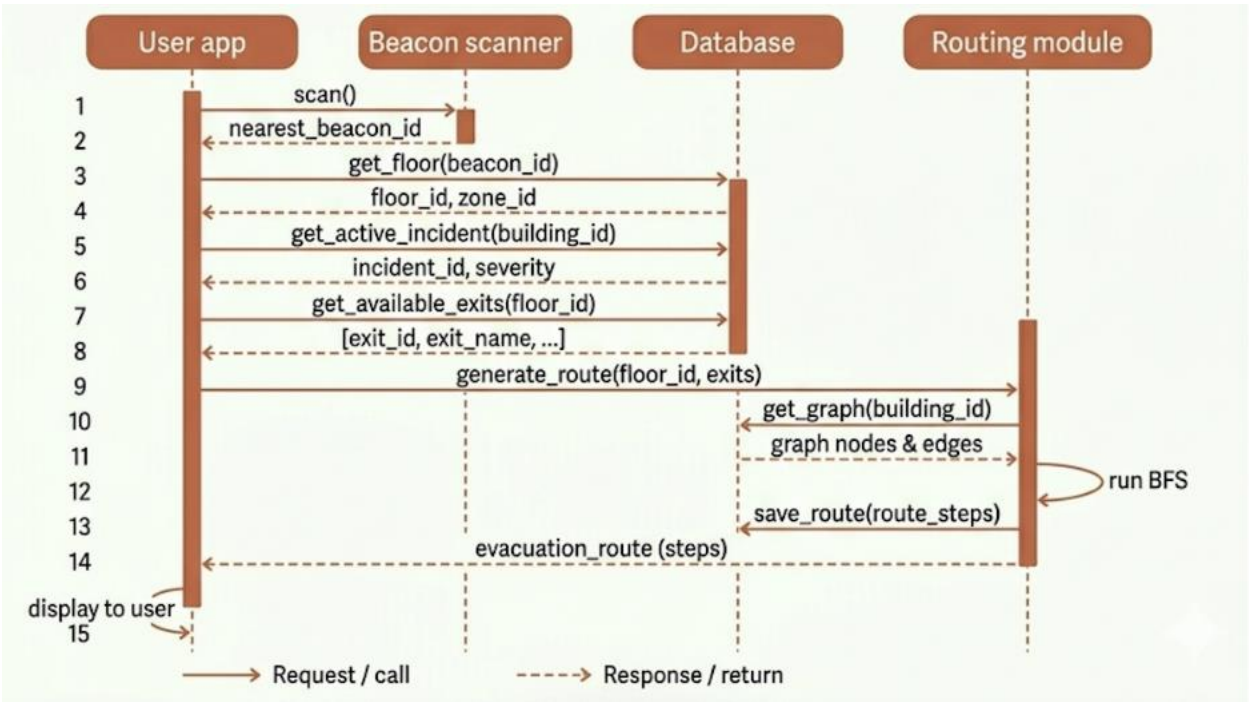


3. Sequence Diagram

The sequence diagram traces all message exchanges between the four system components during a live evacuation: the User App, the Beacon Hardware, the Database, and the Routing Module. Solid arrows represent requests or calls; dashed arrows represent responses or return values.

Figure 4

Sequence Diagram



- User opens app - triggers beacon scan
- Device detects nearest beacon - UUID sent to server
- Server queries beacon-to-floor mapping - floor resolved
- UserLocation record created with timestamp
- Active incident queried for the building
- Blocked exits retrieved and filtered out
- BFS algorithm executed on building graph
- Shortest safe path computed to nearest available exit
- Route record created in database
- Route steps generated and stored

- Steps returned to Routing Module
- Ordered step list sent to User App
- App displays step-by-step navigation to user
- User reaches exit - route marked as completed
- Session logged for post-incident admin review

2. Logical Design

The logical design phase defines how the Smart Evacuation Navigation System's data will be structured in a relational database. It specifies tables, attributes, keys, and relationships while ensuring data integrity and consistency. This phase translates the conceptual model into an organized structure that supports all system operations and queries, illustrated with ER diagrams and relational schemas.

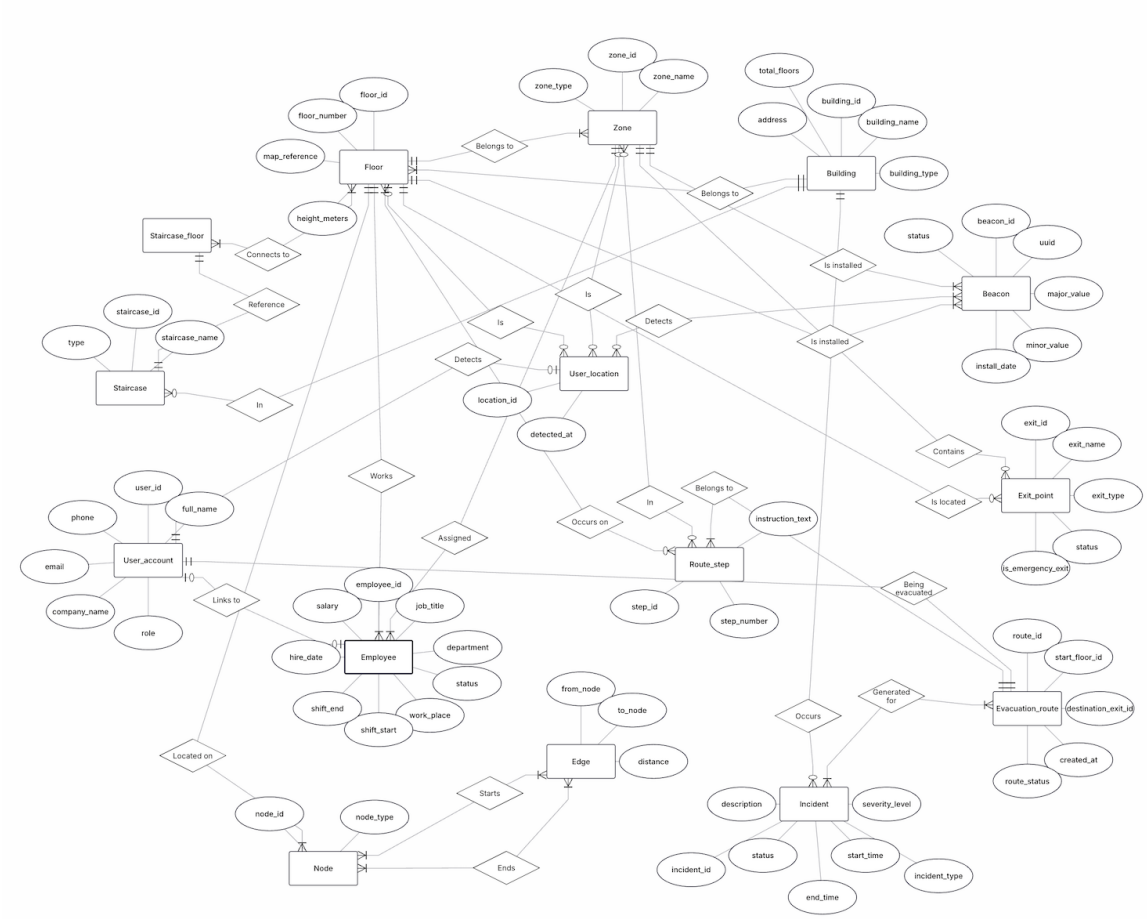
2.1 Entity-Relationship (ER) Model

The system is modelled using an Entity-Relationship (ER) approach to represent the structure of data and all relationships between entities in the Smart Evacuation Navigation System. The ER model organises entities into five functional groups:

Category	Entities	Purpose
Building Structure	Building, Floor, Zone, Staircase	Physical layout of the building
Hardware Components	Beacon	BLE devices used for user positioning
Safety Infrastructure	Exit Point	Exits and emergency escape routes
Users & Tracking	User Account, Employee, User Location	People, roles, and movement tracking
Emergency Management	Incident, Evacuation Route, Route Step	Events, computed paths, and step guidance
Graph Representation	Node, Edge	Navigation graph for BFS pathfinding

Figure 5

ER Diagram



2.2 Entities, Attributes, and Relationships

Each entity is presented below with its full attribute list, data type, constraint, and description. Entities are grouped by a functional category.

Category 1: Building Structure

BUILDING

Attribute	Type	Constraint	Description
building_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for each building
building_name	VARCHAR	**NOT NULL**	Official name of the building
address	VARCHAR	**NOT NULL**	Physical street address
building_type	VARCHAR	**NOT NULL**	Type: mall, hospital, office, etc.
total_floors	INT	**NOT NULL, CHECK > 0**	Total number of floors in the building

FLOOR

Attribute	Type	Constraint	Description
floor_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the floor
building_id	INT	**FK → BUILDING, NOT NULL**	The building this floor belongs to
floor_number	INT	**NOT NULL, UNIQUE per building**	Floor level number (can be 0 for ground)
map_reference	VARCHAR(120)	**NULLABLE**	Reference code to the floor map image
height_meters	DECIMAL(5,2)	**NULLABLE**	Ceiling height in metres

ZONE

Attribute	Type	Constraint	Description
zone_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the zone
floor_id	INT	**FK → FLOOR, NOT NULL**	The floor this zone belongs to
zone_name	VARCHAR	**NOT NULL**	Descriptive name (e.g., Food Court, Ward A)
zone_type	VARCHAR	**NOT NULL**	Type: lobby, room, hallway, stairwell, etc.

STAIRCASE

Attribute	Type	Constraint	Description
staircase	INT	**PK, AUTO_INCREMENT**	Unique identifier for the staircase
building_id	INT	**FK → BUILDING, NOT NULL**	The building the staircase is in
staircase_name	VARCHAR	**NOT NULL**	Name or label (e.g., Staircase A)
type	VARCHAR	**NOT NULL**	Type: emergency, service, public

STAIRCASE_FLOOR (Junction)

Attribute	Type	Constraint	Description
staircase_id	INT	**FK → STAIRCASE, PK (composite)**	References the staircase
floor_id	INT	**FK → FLOOR, PK (composite)**	References the floor the staircase connects to

Category 2: Hardware Components

BEACON

Attribute	Type	Constraint	Description
beacon_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the beacon
floor_id	INT	**FK → FLOOR, NOT NULL**	Floor where the beacon is installed
zone_id	INT	**FK → ZONE, NULLABLE**	Zone where the beacon is installed (optional)
uuid	VARCHAR	**UNIQUE, NOT NULL**	Globally unique BLE identifier string
major_value	INT	**NOT NULL**	BLE major grouping value
minor_value	INT	**NOT NULL**	BLE minor specific value
install_date	DATE	**NOT NULL**	Date the beacon was physically installed
status	ENUM	**active / inactive**	Current operational status of the beacon

Category 3: Safety Infrastructure

EXIT_POINT

Attribute	Type	Constraint	Description
exit_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the exit
floor_id	INT	**FK → FLOOR, NOT NULL**	Floor where this exit is located
zone_id	INT	**FK → ZONE, NULLABLE**	Zone containing this exit (optional)
exit_name	VARCHAR	**NOT NULL**	Descriptive name (e.g., Main Entrance)
exit_type	VARCHAR	**NOT NULL**	Type: door, fire escape, stairwell exit
is_emergency_exit	TINYINT(1)	**NOT NULL, DEFAULT 0**	1 = designated emergency exit, 0 = regular
status	ENUM	**available / blocked**	Current accessibility of this exit

Category 4: Users & Tracking

USER_ACCOUNT

Attribute	Type	Constraint	Description
user_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the user
full_name	VARCHAR	**NOT NULL**	Full name of the user
phone	VARCHAR	**NULLABLE**	Contact phone number
email	VARCHAR	**UNIQUE, NOT NULL**	Login email — must be unique system-wide
role	ENUM	**admin / occupant**	Determines system access and permissions
company_name	VARCHAR	**NULLABLE**	The organisation the user belongs to

EMPLOYEE

Attribute	Type	Constraint	Description
employee_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the employee record
user_id	INT	**FK → USER_ACCOUNT, NOT NULL**	Links to the user's account
building_id	INT	**FK → BUILDING, NOT NULL**	The building where the employee works
zone_id	INT	**FK → ZONE, NULLABLE**	Specific zone assignment (optional)
job_title	VARCHAR(100)	**NOT NULL**	Role title (cashier, security, nurse, etc.)
department	VARCHAR(100)	**NOT NULL**	Departmental group (retail, medical, IT, etc.)
work_place	VARCHAR(150)	**NULLABLE**	Specific work location or shop name

shift_start	TIME	**NULLABLE**	Start time of regular shift
shift_end	TIME	**NULLABLE**	End time of regular shift
hire_date	DATE	**NOT NULL**	Date the employee was hired
salary	DECIMAL(10,2)	**NOT NULL, CHECK >= 0**	Monthly salary in local currency
status	ENUM	**active / inactive**	Current employment status

USER_LOCATION

Attribute	Type	Constraint	Description
location_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the detection event
user_id	INT	**FK → USER_ACCOUNT, NOT NULL**	User who was detected
beacon_id	INT	**FK → BEACON, NOT NULL**	Beacon that detected the user
floor_id	INT	**FK → FLOOR, NOT NULL**	Floor resolved from the beacon
zone_id	INT	**FK → ZONE, NULLABLE**	Zone resolved from the beacon (if assigned)
detected_at	DATETIME	**NOT NULL, DEFAULT NOW()**	Exact timestamp of the detection event

Category 5: Emergency Management

INCIDENT

Attribute	Type	Constraint	Description
incident_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the incident
building_id	INT	**FK → BUILDING, NOT NULL**	The building where the incident occurred
incident_type	VARCHAR	**NOT NULL**	Type: fire, gas leak, flood, security
severity_level	ENUM	**low/medium/high/critical**	Severity classification of the incident
start_time	DATETIME	**NOT NULL**	Time the incident was registered
end_time	DATETIME	**NULLABLE**	Time the incident was resolved (null if active)
status	ENUM	**active / resolved**	Current status of the incident
description	TEXT	**NULLABLE**	Free-text description of the incident

EVACUATION_ROUTE

Attribute	Type	Constraint	Description
route_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the route
incident_id	INT	**FK → INCIDENT, NOT NULL**	The incident this route was generated for
user_id	INT	**FK → USER_ACCOUNT, NOT NULL**	The user being evacuated
start_floor_id	INT	**FK → FLOOR, NOT NULL**	Floor the user was on when route was generated
destination_exit_id	INT	**FK → EXIT_POINT, NOT NULL**	Target exit determined by BFS algorithm

created_at	DATETIME	**NOT NULL, DEFAULT NOW()**	Timestamp when the route was generated
route_status	ENUM	**generated/completed/abandoned**	Current status of the evacuation route

ROUTE_STEP

Attribute	Type	Constraint	Description
step_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the step
route_id	INT	**FK → EVACUATION_ROUTE, NOT NULL**	The route this step belongs to
step_number	INT	**NOT NULL, UNIQUE per route**	Order of this step within the route
floor_id	INT	**FK → FLOOR, NOT NULL**	Floor this navigation step occurs on
zone_id	INT	**FK → ZONE, NULLABLE**	Zone for this step (optional)
instruction_text	TEXT	**NOT NULL**	Plain-text navigation instruction for the user

Category 6: Graph Representation

NODE

Attribute	Type	Constraint	Description
node_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the graph node
floor_id	INT	**FK → FLOOR, NOT NULL**	Floor this node is located on
node_type	VARCHAR	**NOT NULL**	Type: room, intersection, exit, lobby

EDGE

Attribute	Type	Constraint	Description
edge_id	INT	**PK, AUTO_INCREMENT**	Unique identifier for the graph edge
from_node	INT	**FK → NODE, NOT NULL**	Starting node of the walkable connection
to_node	INT	**FK → NODE, NOT NULL**	Ending node of the walkable connection
distance	DECIMAL	**NOT NULL, CHECK > 0**	Physical distance of this path in metres

2.3 Relationship Cardinalities and Constraints

This section defines all relationships between entities in the Smart Evacuation Navigation System. For each relationship the table shows: the two entities involved, the cardinality (how many of one relate to how many of the other), the foreign key constraint that enforces it in SQL, and a plain-language explanation of what the relationship means in the real system.

Relationship	Cardinality	Key Constraint	Explanation
Building Structure			
BUILDING → FLOOR	**(1:M)**	**floor.building_id NOT NULL; FK → BUILDING**	One building contains many floors. A floor cannot exist without a building.
FLOOR → ZONE	**(1:M)**	**zone.floor_id NOT NULL; FK → FLOOR**	One floor is divided into many zones (lobbies, rooms, hallways). A zone belongs to exactly one floor.
BUILDING → STAIRCASE	**(1:M)**	**staircase.building_id NOT NULL; FK → BUILDING**	One building has many staircases. Each staircase is physically located in one building.
STAIRCASE — FLOOR	**(M:M)**	**STAIRCASE_FLOOR: PK(staircase_id, floor_id); both FKs NOT NULL**	A staircase spans multiple floors and a floor can have multiple staircases. Resolved via junction table.
Physical & Navigation Elements			
FLOOR → BEACON	**(1:M)**	**beacon.floor_id NOT NULL; FK → FLOOR**	One floor hosts many BLE beacons. Every beacon must

			be assigned to a specific floor.
ZONE →BEACON	**(1:M)**	**beacon.zone_id NULLABLE; FK → ZONE**	A beacon may optionally be assigned to a zone within its floor. zone_id is nullable.
FLOOR → EXIT_POINT	**(1:M)**	**exit_point.floor_id NOT NULL; FK → FLOOR**	One floor has many exits. Each exit belongs to exactly one floor.
ZONE → EXIT_POINT	**(1:M)**	**exit_point.zone_id NULLABLE; FK → ZONE**	An exit may optionally be located within a specific zone. zone_id is nullable.
User & Employee			
USER_ACCOUNT → USER_LOCATION	**(1:M)**	**user_location.user_id NOT NULL; FK → USER_ACCOUNT**	One user accumulates many location detection records over time. Each record captures one moment.
USER_ACCOUNT → EVACUATION_ROUTE	**(1:M)**	**evacuation_route.user_id NOT NULL; FK → USER_ACCOUNT**	One user can have many evacuation routes generated across different incidents.
USER_ACCOUNT — EMPLOYEE	**(1: 0 or 1)**	**employee.user_id NOT NULL UNIQUE; FK → USER_ACCOUNT**	A user account may optionally have one linked employee record with HR details. Not all users are employees.

BUILDING → EMPLOYEE	**(1:M)**	**employee.building_id NOT NULL; FK → BUILDING**	One building employs many staff. Each employee is assigned to exactly one building.
ZONE → EMPLOYEE	**(1:M)**	**employee.zone_id NULLABLE; FK → ZONE**	An employee may optionally be assigned to a specific zone. zone_id is nullable.
Incident & Evacuation			
BUILDING → INCIDENT	**(1:M)**	**incident.building_id NOT NULL; FK → BUILDING**	One building can have many incidents over time. Each incident is registered against one building.
INCIDENT → EVACUATION_ROUTE	**(1:M)**	**evacuation_route.incident_id NOT NULL; FK → INCIDENT**	One incident triggers many evacuation routes — one per user being evacuated.
EVACUATION_ROUTE → ROUTE_STEP	**(1:M)**	**route_step.route_id NOT NULL; FK → EVACUATION_ROUTE; UNIQUE(route_id, step_number)**	One route is broken into many ordered steps. Step numbers are unique within each route.
Tracking & Navigation Graph			
FLOOR → NODE	**(1:M)**	**node.floor_id NOT NULL; FK → FLOOR**	One floor contains many graph nodes (rooms, intersections, exits). Nodes form the navigation map.

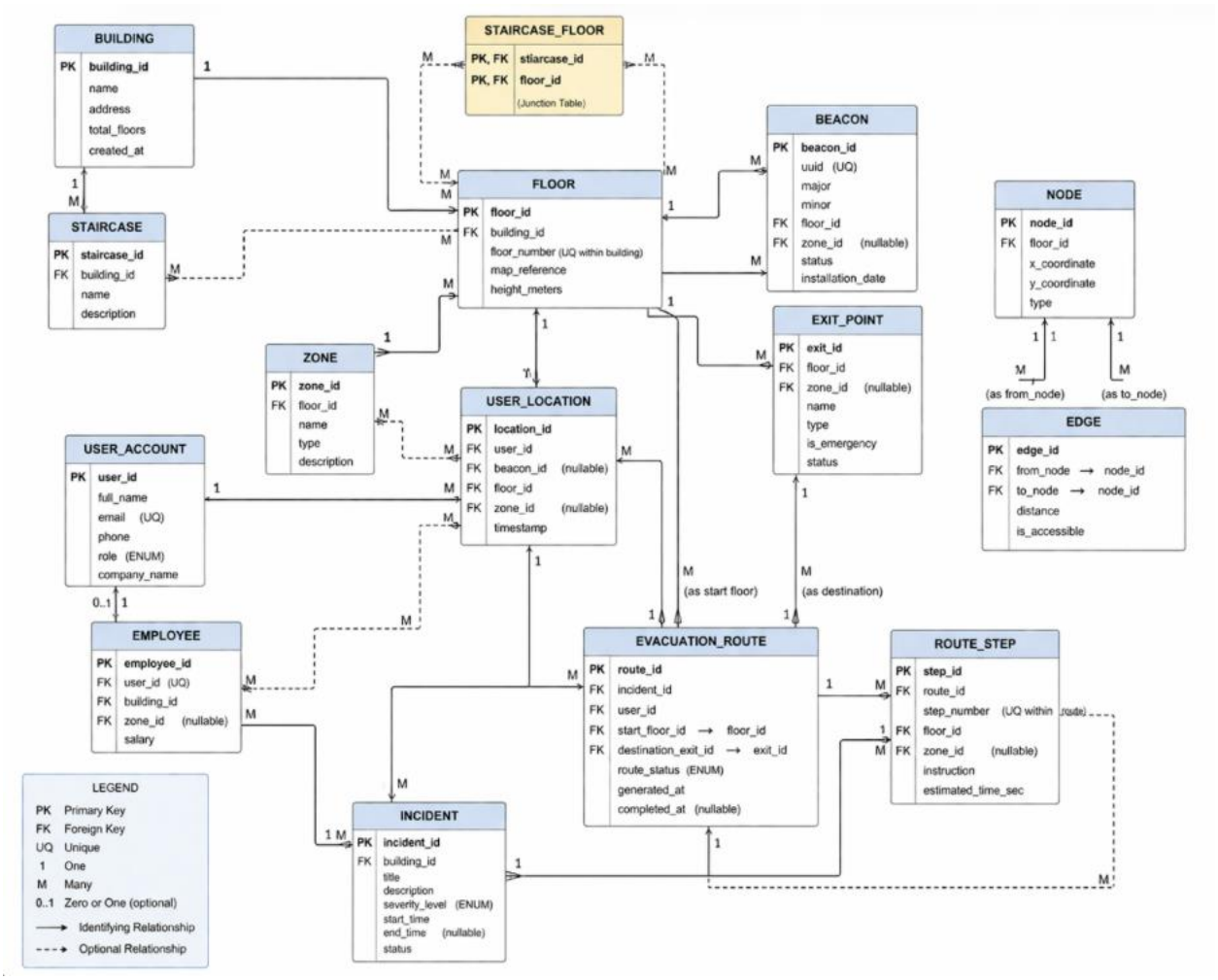
NODE → EDGE (source)	**(1:M)**	**edge.from_node NOT NULL; FK → NODE**	One node is the starting point of many edges — each edge is a walkable path leaving that node.
NODE → EDGE (destination)	**(1:M)**	**edge.to_node NOT NULL; FK → NODE**	One node is also the endpoint of many incoming edges. BFS traverses from → to edges to find the shortest path.

2.4 Relational Scheme

The following relational algebra operations demonstrate how data can be queried and combined from the database and the application of the six core relational algebra operations: Selection (σ), Projection (π), Natural Join (⋈), Union (∪), Intersection (∩), and Difference (−). Each query is presented with a plain-language description, formal relational algebra expression, and the equivalent SQL statement.

Figure 6

Relational Scheme



2.5 Tables, Primary Keys & Foreign Keys

Each table is presented below showing every column, its role (Primary Key, Foreign Key, Unique, or plain attribute), and the table it references when applicable.

BUILDING

Column	Role	References
building_id	**Primary Key**	—

FLOOR

Column	Role	References
floor_id	**Primary Key**	—
building_id	**Foreign Key**	BUILDING(building_id)

ZONE

Column	Role	References
zone_id	**Primary Key**	—
floor_id	**Foreign Key**	FLOOR(floor_id)

BEACON

Column	Role	References
beacon_id	**Primary Key**	—
floor_id	**Foreign Key**	FLOOR(floor_id)
zone_id	**Foreign Key**	ZONE(zone_id) — nullable

EXIT_POINT

Column	Role	References
exit_id	**Primary Key**	—
floor_id	**Foreign Key**	FLOOR(floor_id)
zone_id	**Foreign Key**	ZONE(zone_id) — nullable

STAIRCASE

Column	Role	References
staircase_id	**Primary Key**	—
building_id	**Foreign Key**	BUILDING(building_id)

STAIRCASE_FLOOR

Column	Role	References
staircase_id	Primary Key + Foreign Key	STAIRCASE(staircase_id)
floor_id	Primary Key + Foreign Key	FLOOR(floor_id)

USER_ACCOUNT

Column	Role	References
user_id	**Primary Key**	—

EMPLOYEE

Column	Role	References
employee_id	**Primary Key**	—
user_id	**Foreign Key**	USER_ACCOUNT(user_id)
building_id	**Foreign Key**	BUILDING(building_id)
zone_id	**Foreign Key**	ZONE(zone_id) — nullable

USER_LOCATION

Column	Role	References
location_id	**Primary Key**	—
user_id	**Foreign Key**	USER_ACCOUNT(user_id)
beacon_id	**Foreign Key**	BEACON(beacon_id)
floor_id	**Foreign Key**	FLOOR(floor_id)
zone_id	**Foreign Key**	ZONE(zone_id) — nullable

INCIDENT

Column	Role	References
incident_id	**Primary Key**	—
building_id	**Foreign Key**	BUILDING(building_id)

EVACUATION_ROUTE

Column	Role	References
route_id	**Primary Key**	—
incident_id	**Foreign Key**	INCIDENT(incident_id)
user_id	**Foreign Key**	USER_ACCOUNT(user_id)
start_floor_id	**Foreign Key**	FLOOR(floor_id)
destination_exit_id	**Foreign Key**	EXIT_POINT(exit_id)

ROUTE_STEP

Column	Role	References
step_id	**Primary Key**	—
route_id	**Foreign Key**	EVACUATION_ROUTE(route_id)
floor_id	**Foreign Key**	FLOOR(floor_id)
zone_id	**Foreign Key**	ZONE(zone_id) — nullable

NODE

Column	Role	References
node_id	**Primary Key**	—
floor_id	**Foreign Key**	FLOOR(floor_id)

EDGE

Column	Role	References
edge_id	**Primary Key**	—
from_node	**Foreign Key**	NODE(node_id)
to_node	**Foreign Key**	NODE(node_id)

2.6 Normalization Process

All tables in the Smart Evacuation Navigation System have been analysed and normalised through three normal forms to eliminate data redundancy and ensure referential integrity.

First Normal Form (1NF)

A table satisfies 1NF when every cell contains a single, indivisible (atomic) value, every column in a row is distinct, and every row can be uniquely identified by a primary key.

This schema satisfies 1NF:

- All attribute values are atomic. For example, the BEACON table stores one uuid per row - if a floor needs multiple beacons, each beacon gets its own row, never a comma-separated list of UUIDs in a single cell.
- There are no repeating column groups. For example, a route does not store step_1, step_2, step_3 as separate columns. Instead, each step is stored as an independent row in ROUTE_STEP with a step_number attribute.
- Multiple exit types or zone types are never stored in one field. Each EXIT_POINT row holds one exit with one status, one type, and one emergency flag.
- Every table has a clearly defined primary key - either a single AUTO_INCREMENT integer (e.g., building_id, user_id, incident_id) or a composite key (e.g., STAIRCASE_FLOOR uses PK(staircase_id, floor_id)) - that uniquely identifies each record.
- The USER_LOCATION table logs each beacon detection as a separate atomic row with its own timestamp, rather than storing a list of detections inside a single user record.

Second Normal Form (2NF)

A table satisfies 2NF when it is already in 1NF and every non-key attribute is fully functionally dependent on the entire primary key — not just a part of it. Partial dependency is only a risk in tables with a composite primary key.

This schema satisfies 2NF:

- The only table with a composite primary key is STAIRCASE_FLOOR (PK: staircase_id + floor_id). This table intentionally contains no non-key attributes - its sole purpose is to record which floors a staircase connects. Since there are no other columns, there can be no partial dependency, and 2NF is trivially satisfied.
- All other tables use a single column surrogate primary key (AUTO_INCREMENT integer). By definition, a table with a single-column PK cannot have a partial dependency - every non-key attribute depends on that one key, so 2NF is automatically satisfied.
- For example, in EVACUATION_ROUTE, every attribute (incident_id, user_id, start_floor_id, destination_exit_id, created_at, route_status) depends fully on route_id. None of them depend on only part of the key, because there is only one key column.
- In ROUTE_STEP, although the combination (route_id, step_number) forms a logical unique constraint, the actual primary key is the surrogate step_id. All attributes - floor_id, zone_id, instruction_text - depend on step_id alone, satisfying 2NF.

Third Normal Form (3NF)

A table satisfies 3NF when it is already in 2NF and no non-key attribute is transitively dependent on the primary key through another non-key attribute. In other words, non-key attributes must depend only on the primary key — not on each other.

This schema satisfies 3NF:

- User personal data (full_name, email, phone, role, company_name) is stored exclusively in USER_ACCOUNT. It is never duplicated in EVACUATION_ROUTE, USER_LOCATION, or EMPLOYEE. Those tables reference user_id as a foreign key instead. This eliminates the transitive dependency: route → user_id → full_name is broken by storing the name only in USER_ACCOUNT.
- Building details (building_name, address, building_type, total_floors) are stored only in BUILDING and referenced by foreign key everywhere else. INCIDENT, FLOOR, EMPLOYEE, and STAIRCASE all store building_id — they do not repeat the building name or address, which would create a transitive dependency.
- Floor metadata (floor_number, height_meters, map_reference) is stored only in FLOOR. BEACON, EXIT_POINT, NODE, ROUTE_STEP, and USER_LOCATION all reference floor_id without duplicating any floor attribute.
- Zone information (zone_name, zone_type) is stored only in ZONE. Tables like BEACON, EXIT_POINT, EMPLOYEE, and ROUTE_STEP store zone_id as a nullable foreign key, never the zone name directly.
- In the EMPLOYEE table, all attributes — job_title, department, work_place, shift_start, shift_end, hire_date, salary, status — depend directly and only on employee_id. None of them are derivable from another non-key attribute. For example, salary does not depend on department; it depends on the specific employee record.
- In INCIDENT, severity_level and incident_type are independent attributes that both depend directly on incident_id. There is no case where one non-key attribute determines another.

Normalization Summary Table

Table	1NF	2NF	3NF	Notes
BUILDING	✓	✓	✓	Single PK, all attributes atomic
FLOOR	✓	✓	✓	FK to BUILDING, no transitive deps
ZONE	✓	✓	✓	FK to FLOOR, zone_type is independent
BEACON	✓	✓	✓	uuid is unique, no derivable attributes
EXIT_POINT	✓	✓	✓	Each exit is a separate atomic row
STAIRCASE	✓	✓	✓	Single PK, FK to BUILDING
STAIRCASE_FLOOR	✓	✓	✓	Composite PK, junction only — no non-key attrs
USER_ACCOUNT	✓	✓	✓	email unique; role is not derived
EMPLOYEE	✓	✓	✓	Separate from USER_ACCOUNT to avoid redundancy
USER_LOCATION	✓	✓	✓	Each detection event is one atomic row
INCIDENT	✓	✓	✓	Building info kept in BUILDING table
EVACUATION_ROUTE	✓	✓	✓	User and incident data not duplicated here
ROUTE_STEP	✓	✓	✓	step_number unique per route via composite unique
NODE	✓	✓	✓	node_type is atomic, no derivation
EDGE	✓	✓	✓	distance is independent, no transitive dep

Conclusion: All 15 tables satisfy 3NF. The schema is fully normalized with no data redundancy, no partial dependencies, and no transitive dependencies.

2.7 Relational Algebra Operations

The following 30 queries illustrate the application of the six relational algebra operations including Selection (σ), Projection (π), Natural Join (⋈), Union (∪), Intersection (∩) and Difference (−). Each query is presented with a plain-language description, its formal relational algebra expression and the equivalent SQL statement.

σ Selection - Filter rows by condition

1. Retrieve all active incidents	
Filters the INCIDENT table to return only records where the status is 'active'. Used by the system to determine which emergencies are currently ongoing.	

Relational Algebra
σ status = 'active' (INCIDENT)
	SQL Equivalent
	`SELECT * FROM INCIDENT`
	`WHERE status = 'active';`

2. Retrieve all floors in a specific building	
Returns every floor that belongs to building with id = 1. Used when loading the full layout of a registered building.	

Relational Algebra
σ building_id = 1 (FLOOR)
	SQL Equivalent
	`SELECT * FROM FLOOR`
	`WHERE building_id = 1;`

3. Retrieve all blocked exits	
Selects all exit points whose current status is 'blocked'. This result is used during BFS pathfinding to exclude unavailable exits from the route.	

Relational Algebra
σ status = 'blocked' (EXIT_POINT)
	SQL Equivalent
	`SELECT * FROM EXIT_POINT`
	`WHERE status = 'blocked';`

4. Retrieve all emergency exits that are available	
Filters EXIT_POINT to return only exits that are both designated as emergency exits and currently accessible. These are the valid targets for evacuation routes.	
Relational Algebra	**SQL Equivalent**

σ is_emergency_exit = 1 ∧ status = 'available' (EXIT_POINT)	`SELECT * FROM EXIT_POINT`
	`WHERE is_emergency_exit = 1`
	`AND status = 'available';`

5. Retrieve all active beacons	
Returns only beacons with status 'active'. Inactive beacons are excluded from floor detection to avoid incorrect location resolution.	
Relational Algebra	**SQL Equivalent**
σ status = 'active' (BEACON)	`SELECT * FROM BEACON` `WHERE status = 'active';`

6. Retrieve all admin users	
Filters USER_ACCOUNT to return only users with the 'admin' role. Admins are the only users permitted to register incidents and modify building structure.	
Relational Algebra	**SQL Equivalent**
σ role = 'admin' (USER_ACCOUNT)	`SELECT * FROM USER_ACCOUNT` `WHERE role = 'admin';`

7. Retrieve all critical severity incidents	
Selects incidents classified as 'critical'. These require immediate attention and trigger priority evacuation route generation.	

Relational Algebra
σ severity_level = 'critical' (INCIDENT)
	SQL Equivalent
	`SELECT * FROM INCIDENT`
	`WHERE severity_level = 'critical';`

π Projection - Select specific columns

8. Get names and addresses of all buildings	
Projects only the building_name and address columns from the BUILDING table. Used to display a summary list of registered buildings.	

Relational Algebra
π building_name, address (BUILDING)
	SQL Equivalent
	`SELECT building_name, address`
	`FROM BUILDING;`

9. Get usernames and emails of all users	
Retrieves only the full_name and email from USER_ACCOUNT, discarding all other attributes. Useful for generating a contact directory.	

Relational Algebra
π full_name, email (USER_ACCOUNT)
	SQL Equivalent
	`SELECT full_name, email`
	`FROM USER_ACCOUNT;`

10. Get beacon UUIDs and their install dates	
Projects the uuid and install_date from BEACON. Used for hardware audit reports to track when each beacon was deployed.	

Relational Algebra
π uuid, install_date (BEACON)
	SQL Equivalent
	`SELECT uuid, install_date`
	`FROM BEACON;`

11. Get all exit names and their types	
Projects exit_name and exit_type from EXIT_POINT. Used to build a reference list of all exits in the system.	
Relational Algebra	**SQL Equivalent**
π exit_name, exit_type (EXIT_POINT)	`SELECT exit_name, exit_type` `FROM EXIT_POINT;`

12. Get incident types and severity levels	
Projects incident_type and severity_level from INCIDENT. Useful for generating a summary report of all emergency event categories.	
Relational Algebra	**SQL Equivalent**
π incident_type, severity_level (INCIDENT)	`SELECT incident_type, severity_level` `FROM INCIDENT;`

⋈ *Join - Combine data from related tables*

13. Get beacons with their floor number and building name	
Joins BEACON with FLOOR and BUILDING to return each beacon alongside the floor number and building it belongs to. Used in the admin dashboard to display beacon locations.	
Relational Algebra	**SQL Equivalent**

π uuid, floor_number, building_name (
BEACON ⋈ FLOOR ⋈ BUILDING)
	`SELECT B.uuid, F.floor_number,`
	`BL.building_name`
	`FROM BEACON B`
	`JOIN FLOOR F ON B.floor_id = F.floor_id`
	`JOIN BUILDING BL ON F.building_id = BL.building_id;`

14. Get all evacuation routes with the user's name and incident type	
Joins EVACUATION_ROUTE with USER_ACCOUNT and INCIDENT to display who was evacuated, during which incident, and what the current route status is.	
Relational Algebra	**SQL Equivalent**

EVACUATION_ROUTE ⋈ USER_ACCOUNT ⋈ INCIDENT	`SELECT UA.full_name, I.incident_type, ER.route_status`
	`FROM EVACUATION_ROUTE ER`
	`JOIN USER_ACCOUNT UA ON ER.user_id = UA.user_id`
	`JOIN INCIDENT I ON ER.incident_id = I.incident_id;`

15. Get all route steps with their floor number and instruction	
Joins ROUTE_STEP with EVACUATION_ROUTE and FLOOR to return the ordered steps of a route alongside the floor they occur on.	
Relational Algebra	**SQL Equivalent**

ROUTE_STEP ⋈ EVACUATION_ROUTE ⋈ FLOOR	`SELECT RS.step_number, F.floor_number, RS.instruction_text`
	`FROM ROUTE_STEP RS`
	`JOIN EVACUATION_ROUTE ER ON RS.route_id = ER.route_id`
	`JOIN FLOOR F ON RS.floor_id = F.floor_id`
	`ORDER BY RS.route_id, RS.step_number;`

16. Get user location history with beacon UUID and floor number	
Joins USER_LOCATION with USER_ACCOUNT, BEACON, and FLOOR to produce the full audit trail — showing who was detected, by which beacon, on which floor, and when.	
Relational Algebra	**SQL Equivalent**
	`SELECT UA.full_name, BC.uuid, F.floor_number, UL.detected_at`

USER_LOCATION ⋈ USER_ACCOUNT ⋈ BEACON ⋈ FLOOR	`FROM USER_LOCATION UL`
	`JOIN USER_ACCOUNT UA ON UL.user_id = UA.user_id`
	`JOIN BEACON BC ON UL.beacon_id = BC.beacon_id`
	`JOIN FLOOR F ON UL.floor_id = F.floor_id;`

17. Get all incidents with the building name and address

Joins INCIDENT with BUILDING to display the full context of each emergency — the building name and address alongside incident type, severity, and status.

Relational Algebra	SQL Equivalent

INCIDENT ⋈ BUILDING	`SELECT I.incident_type, I.severity_level, I.status,`
	`  B.building_name, B.address`
	`FROM INCIDENT I`
	`JOIN BUILDING B ON I.building_id = B.building_id;`

18. Get employees with their user details and building name

Joins EMPLOYEE with USER_ACCOUNT and BUILDING to display each employee's full name, job title, department, and the building they work in.

Relational Algebra	SQL Equivalent

EMPLOYEE ⋈ USER_ACCOUNT ⋈ BUILDING	`SELECT UA.full_name, E.job_title, E.department,`
	`  B.building_name`
	`FROM EMPLOYEE E`
	`JOIN USER_ACCOUNT UA ON E.user_id = UA.user_id`
	`JOIN BUILDING B ON E.building_id = B.building_id;`

19. Get all exits with their floor number and zone name	
Joins EXIT_POINT with FLOOR and ZONE to show the exact physical location of each exit — the floor it is on and the zone it belongs to.	
Relational Algebra	**SQL Equivalent**

EXIT_POINT ⋈ FLOOR ⋈ ZONE	`SELECT EP.exit_name, EP.exit_type, F.floor_number,`
	`Z.zone_name`
	`FROM EXIT_POINT EP`
	`JOIN FLOOR F ON EP.floor_id = F.floor_id`
	`LEFT JOIN ZONE Z ON EP.zone_id = Z.zone_id;`

∪ *Union - Combine results from two compatible sets*

20. Get all users — both admins and occupants	
Returns the full_name and email of all users regardless of role, by taking the union of the admin set and the occupant set. The result contains every user in the system.	
Relational Algebra	**SQL Equivalent**

π full_name, email (σ role='admin' (USER_ACCOUNT))
∪
π full_name, email (σ role='occupant' (USER_ACCOUNT))
	`SELECT full_name, email FROM USER_ACCOUNT WHERE role = 'admin'`
	`UNION`
	`SELECT full_name, email FROM USER_ACCOUNT WHERE role = 'occupant';`

21. Get all exits that are either available or designated as emergency exits

Returns any exit that meets at least one of the two conditions: it is currently open, or it is an emergency exit. Useful for building a shortlist of viable evacuation targets.

Relational Algebra	SQL Equivalent

σ status='available' (EXIT_POINT)
∪
σ is_emergency_exit=1 (EXIT_POINT)
	`SELECT * FROM EXIT_POINT WHERE status = 'available'`
	`UNION`
	`SELECT * FROM EXIT_POINT WHERE is_emergency_exit = 1;`

22. Get all incidents that are either active or critical

Combines the set of currently active incidents with the set of critical incidents. An incident appears in the result if it satisfies either condition.

Relational Algebra	SQL Equivalent

σ status='active' (INCIDENT)
∪
σ severity_level='critical' (INCIDENT)
	`SELECT * FROM INCIDENT WHERE status = 'active'`
	`UNION`
	`SELECT * FROM INCIDENT WHERE severity_level = 'critical';`

∩ *Intersection - Rows satisfying both conditions*

23. Get exits that are both available AND designated as emergency exits

Returns only the exits that satisfy both conditions simultaneously — currently open and officially designated as emergency exits. These are the highest-priority BFS targets.

Relational Algebra	SQL Equivalent

σ status='available' (EXIT_POINT)
∩
σ is_emergency_exit=1 (EXIT_POINT)
	`SELECT * FROM EXIT_POINT WHERE status = 'available'`
	`INTERSECT`
	`SELECT * FROM EXIT_POINT WHERE is_emergency_exit = 1;`

24. Get incidents that are both active AND high severity

Returns only incidents that are currently active and classified as high severity. These are the most urgent events requiring immediate evacuation route generation.

Relational Algebra	SQL Equivalent

σ status='active' (INCIDENT)
∩
σ severity_level='high' (INCIDENT)
	`SELECT * FROM INCIDENT WHERE status = 'active'`
	`INTERSECT`
	`SELECT * FROM INCIDENT WHERE severity_level = 'high';`

25. Get admin users belonging to a specific company

Returns only user accounts that are both admins and members of 'Grand Mall Mgmt'. Used to identify which admins are authorised for a specific building.

Relational Algebra	SQL Equivalent

σ role='admin' (USER_ACCOUNT)
∩
σ company_name='Grand Mall Mgmt' (USER_ACCOUNT)
	`SELECT * FROM USER_ACCOUNT WHERE role = 'admin'`
	`INTERSECT`
	`SELECT * FROM USER_ACCOUNT WHERE company_name = 'Grand Mall Mgmt';`

– *Difference - Rows in first set but NOT in second*

26. Get all users who have never been assigned an evacuation route

Returns users who exist in USER_ACCOUNT but have no corresponding record in EVACUATION_ROUTE. These are users who were never evacuated during any incident.

Relational Algebra	SQL Equivalent

π user_id (USER_ACCOUNT)
–
π user_id (EVACUATION_ROUTE)
	`SELECT user_id FROM USER_ACCOUNT`
	`EXCEPT`
	`SELECT user_id FROM EVACUATION_ROUTE;`

27. Get exits that are not blocked

Returns all exit IDs that exist in EXIT_POINT but are not in the set of blocked exits. Equivalent to filtering for available exits without using a direct status filter.

Relational Algebra	SQL Equivalent

π exit_id (EXIT_POINT)
–
π exit_id (σ status='blocked' (EXIT_POINT))
	`SELECT exit_id FROM EXIT_POINT`
	`EXCEPT`
	`SELECT exit_id FROM EXIT_POINT WHERE status = 'blocked';`

28. Get incidents that have not been resolved

Returns all incident IDs minus those that have been resolved. These are incidents still requiring active management.

Relational Algebra	SQL Equivalent

π incident_id (INCIDENT)
–
π incident_id (σ status='resolved' (INCIDENT))
	`SELECT incident_id FROM INCIDENT`
	`EXCEPT`
	`SELECT incident_id FROM INCIDENT WHERE status = 'resolved';`

29. Get floors that have no beacon installed	
Returns floor IDs from FLOOR that do not appear in the BEACON table. These floors have no positioning hardware and cannot support user location detection.	
Relational Algebra	**SQL Equivalent**;
π floor_id (FLOOR)	`SELECT floor_id FROM FLOOR`
–	`EXCEPT`
π floor_id (BEACON)	`SELECT floor_id FROM BEACON;`

30. Get users with no location detection record	
Returns user IDs that exist in USER_ACCOUNT but have never appeared in USER_LOCATION. These users have never opened the app during an incident.	
Relational Algebra	**SQL Equivalent**
π user_id (USER_ACCOUNT)	`SELECT user_id FROM USER_ACCOUNT`
–	`EXCEPT`
π user_id (USER_LOCATION)	`SELECT user_id FROM USER_LOCATION;`

3. Physical Implementation

This section documents the complete physical implementation of the Smart Evacuation Navigation System database using MySQL Server. It translates the logical schema from Section 2 into executable SQL, covering database creation, table definitions, data insertion, query scripts, views, indexes, triggers, stored procedures, access control, deployment, and project conclusions.

3.1 DDL - Database and Table Creation

DDL (Data Definition Language) creates the structure of the database. The database uses UTF-8 encoding to support international characters. All 15 tables are defined with named constraints that match the logical schema exactly from Section 2.5.

Table Creation

The following 15 tables were defined. Each table is shown with its purpose, primary key, foreign keys, and additional constraints.

Database Creation

```
IF EXISTS (SELECT name FROM sys.databases WHERE name = 'SmartEvacuationDB')
BEGIN
   ALTER DATABASE SmartEvacuationDB SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;
   DROP DATABASE SmartEvacuationDB;
END
GO

CREATE DATABASE SmartEvacuationDB;
GO
USE SmartEvacuationDB;
GO
```

Sample DDL - INCIDENT Table

Below is the INCIDENT table definition, which demonstrates the use of named constraints, CHECK constraints, and T-SQL IDENTITY columns.

```
CREATE TABLE INCIDENT (
    incident_id    INT          NOT NULL IDENTITY(1,1),
    building_id    INT          NOT NULL,
    incident_type  VARCHAR(80)   NOT NULL,
    severity_level VARCHAR(10)   NOT NULL,
    start_time     DATETIME      NOT NULL,
    end_time       DATETIME      NULL,
    status         VARCHAR(10)   NOT NULL DEFAULT 'active',
    description    NVARCHAR(MAX) NULL,
    CONSTRAINT pk_incident        PRIMARY KEY (incident_id),
    CONSTRAINT fk_inc_building    FOREIGN KEY (building_id) REFERENCES
BUILDING(building_id),
    CONSTRAINT chk_incident_times CHECK (end_time IS NULL OR end_time > start_time),
    CONSTRAINT chk_inc_severity   CHECK (severity_level IN
('low','medium','high','critical')),
    CONSTRAINT chk_inc_status     CHECK (status IN ('active','resolved'))
);
GO
```

3.2 DML - Data Insertion

DML (Data Manipulation Language) populates all 15 tables with realistic test data. Every table contains 40 or more records. The data simulates a real multi-building deployment with buildings across Yerevan, Gyumri, Vanadzor, and Abovyan, Armenia.

Record Counts per Table

Table	Records	Notes
BUILDING	**40+**	Buildings of different types: shopping malls, hospitals, offices, hotels, stadiums, universities
FLOOR	**45+**	5 floors per building for buildings 1–10; additional floors for buildings 1 and 2
ZONE	**50+**	5–6 named zones per building: lobby, room, hallway, parking, storage
BEACON	**50+**	BLE beacons distributed on floors and zones; mix of active and inactive
EXIT_POINT	**50+**	Emergency exits and regular doors; some blocked to simulate real incident conditions
STAIRCASE	**40+**	One or two staircases per building (emergency and service types)
STAIRCASE_FLOOR	**42+**	Junction records connecting each staircase to its floors
USER_ACCOUNT	**45**	9 admins + 36 occupants; all unique emails
EMPLOYEE	**45**	One record per user; UNIQUE(user_id) enforces 1:0..1 relationship
INCIDENT	**45+**	Fire, gas leak, flood, security events; active and resolved; all severity levels

EVACUATION_ROUTE	**45+**	Generated, completed, and abandoned routes across all incident types
ROUTE_STEP	**45+**	2–3 ordered navigation steps per route with plain-language instructions
USER_LOCATION	**45+**	Beacon detection events timestamped during incident periods
NODE	**45**	One graph node per floor: lobby, room, intersection, exit
EDGE	**45**	Walkable paths between nodes with distances in metres; all > 0

Sample DML Statement

```
INSERT INTO BUILDING (building_name, address, building_type, total_floors) VALUES
('Grand Mall',        '12 Commerce Blvd, Yerevan',   'Shopping Mall', 5),
('Tower One',         '45 Republic Ave, Yerevan',    'Office Tower',  5),
('City Hospital',     '78 Health St, Yerevan',       'Hospital',      5),
('State University',  '200 Education Rd, Gyumri',    'University',    5),
('Zvartnots Airport', 'Airport Highway, Yerevan',    'Airport',       5);
GO
```

3.3 DQL - Data Retrieval Queries

The 33 SQL queries below are the direct T-SQL implementation of the 33 Relational Algebra operations from Section 2.6 of the Logical Design. The query number, operation type, and result set are identical in both documents.

Selection - Q01 to Q10 (σ)

Selection filters rows from a table based on a condition.

#	Description	T-SQL
Q01	All active incidents	SELECT * FROM INCIDENT WHERE status = 'active';
Q02	All floors in building 1	SELECT * FROM FLOOR WHERE building_id = 1;
Q03	All blocked exits	SELECT * FROM EXIT_POINT WHERE status = 'blocked';
Q04	Available emergency exits	SELECT * FROM EXIT_POINT WHERE is_emergency_exit=1 AND status='available';
Q05	All active beacons	SELECT * FROM BEACON WHERE status = 'active';
Q06	All admin users	SELECT * FROM USER_ACCOUNT WHERE role = 'admin';
Q07	Critical incidents	SELECT * FROM INCIDENT WHERE severity_level = 'critical';
Q08	All zones on floor 3	SELECT * FROM ZONE WHERE floor_id = 3;
Q09	Completed evacuation routes	SELECT * FROM EVACUATION_ROUTE WHERE route_status = 'completed';
Q10	Active employees	SELECT * FROM EMPLOYEE WHERE status = 'active';

Projection - Q11 to Q15 (π)

Projection selects specific columns only.

#	Description	T-SQL
Q11	Building names and addresses	SELECT building_name, address FROM BUILDING;
Q12	User names and emails	SELECT full_name, email FROM USER_ACCOUNT;
Q13	Beacon UUIDs and install dates	SELECT uuid, install_date FROM BEACON;
Q14	Exit names and types	SELECT exit_name, exit_type FROM EXIT_POINT;
Q15	Incident types and severities	SELECT incident_type, severity_level FROM INCIDENT;

Join - Q16 to Q22 (⋈)

Join combines related tables using foreign key matches.

Q16 - Beacons with floor number and building

```
SELECT bc.uuid, f.floor_number, b.building_name
FROM   BEACON bc
JOIN   FLOOR f    ON bc.floor_id   = f.floor_id
JOIN   BUILDING b ON f.building_id = b.building_id;
```

Q17 - Routes with user name and incident type

```
SELECT ua.full_name, i.incident_type, i.severity_level, er.route_status
FROM   EVACUATION_ROUTE er
JOIN   USER_ACCOUNT ua ON er.user_id     = ua.user_id
JOIN   INCIDENT i      ON er.incident_id = i.incident_id;
```

Q18 - Route steps with floor number and instruction

```
SELECT rs.step_number, f.floor_number, rs.instruction_text
FROM   ROUTE_STEP rs
```

```
JOIN   EVACUATION_ROUTE er ON rs.route_id = er.route_id
JOIN   FLOOR f             ON rs.floor_id = f.floor_id
ORDER  BY rs.route_id, rs.step_number;
```

Q20 - Incidents with building name and address

```
SELECT i.incident_type, i.severity_level, i.status, b.building_name, b.address
FROM   INCIDENT i JOIN BUILDING b ON i.building_id = b.building_id;
```

Q21 - Employees with user details and building

```
SELECT ua.full_name, e.job_title, e.department, b.building_name
FROM   EMPLOYEE e
JOIN   USER_ACCOUNT ua ON e.user_id     = ua.user_id
JOIN   BUILDING b       ON e.building_id = b.building_id;
```

Q22 - Exits with floor, zone, and building

```
SELECT ep.exit_name, ep.status, f.floor_number,
       COALESCE(z.zone_name,'N/A') AS zone_name, b.building_name
FROM   EXIT_POINT ep
JOIN   FLOOR f    ON ep.floor_id   = f.floor_id
JOIN   BUILDING b ON f.building_id = b.building_id
LEFT   JOIN ZONE z ON ep.zone_id   = z.zone_id;
```

Union - Q23 to Q25 (∪)

Q23 - All users (admins U occupants)

```
SELECT full_name, email FROM USER_ACCOUNT WHERE role = 'admin'
UNION
SELECT full_name, email FROM USER_ACCOUNT WHERE role = 'occupant';
```

Q24 - Exits that are available U emergency exits

```
SELECT * FROM EXIT_POINT WHERE status = 'available'
UNION
SELECT * FROM EXIT_POINT WHERE is_emergency_exit = 1;
```

Q25 - Incidents that are active U critical severity

```
SELECT * FROM INCIDENT WHERE status = 'active'
```

```
UNION
SELECT * FROM INCIDENT WHERE severity_level = 'critical';
```

Intersection - Q26 to Q28 (∩)

SQL Server supports INTERSECT natively - used directly.

Q26 - Exits that are BOTH available AND emergency exits

```
SELECT * FROM EXIT_POINT WHERE status = 'available'
INTERSECT
SELECT * FROM EXIT_POINT WHERE is_emergency_exit = 1;
```

Q27 - Incidents that are BOTH active AND high severity

```
SELECT * FROM INCIDENT WHERE status = 'active'
INTERSECT
SELECT * FROM INCIDENT WHERE severity_level = 'high';
```

Q28 - Admin users belonging to Grand Mall Mgmt

```
SELECT * FROM USER_ACCOUNT WHERE role = 'admin'
INTERSECT
SELECT * FROM USER_ACCOUNT WHERE company_name = 'Grand Mall Mgmt';
```

Difference - Q29 to Q33 (−)

SQL Server supports EXCEPT natively — used directly.

Q29 - Users never assigned an evacuation route

```
SELECT user_id FROM USER_ACCOUNT
EXCEPT
SELECT user_id FROM EVACUATION_ROUTE;
```

Q30 - Exits that are NOT blocked

```
SELECT exit_id FROM EXIT_POINT
EXCEPT
SELECT exit_id FROM EXIT_POINT WHERE status = 'blocked';
```

Q31 - Incidents not yet resolved

```
SELECT incident_id FROM INCIDENT
EXCEPT
SELECT incident_id FROM INCIDENT WHERE status = 'resolved';
```

Q32 - Floors with no beacon installed

```
SELECT floor_id FROM FLOOR  EXCEPT  SELECT floor_id FROM BEACON;
```

Q33 - Users with no location detection record

```
SELECT user_id FROM USER_ACCOUNT  EXCEPT  SELECT user_id FROM
USER_LOCATION;
```

3.4 Views

Seven views are defined using CREATE OR ALTER VIEW syntax. Views encapsulate complex queries, provide reusable data access, and form a security layer — the evac_readonly user can query views without needing direct table access.

View Name	Query Logic	Columns Returned	Used By
vw_active_incidents	INCIDENT ⋈ BUILDING WHERE status='active'	incident_id, building_name, address, incident_type, severity_level, start_time, description	Admin dashboard — displays all ongoing emergencies.
vw_available_emergency_exits	EXIT_POINT ⋈ FLOOR ⋈ BUILDING WHERE emergency=1 AND available	exit_id, exit_name, exit_type, floor_id, floor_number, building_name	BFS routing engine — provides valid evacuation targets.
vw_evacuation_details	EVACUATION_ROUTE ⋈ USER_ACCOUNT ⋈ INCIDENT ⋈ BUILDING ⋈ FLOOR ⋈ EXIT_POINT LEFT JOIN ROUTE_STEP	route_id, evacuee, building_name, incident_type, start_floor, destination_exit, route_status, step_number, instruction	Admin post-incident review — full picture of every evacuation.
vw_beacon_summary	BEACON ⋈ FLOOR ⋈ BUILDING LEFT JOIN ZONE	beacon_id, uuid, status, install_date, floor_number,	Hardware management — shows all beacon

		building_name, zone_name	locations and statuses.
vw_employee_directory	EMPLOYEE ⋈ USER_ACCOUNT ⋈ BUILDING LEFT JOIN ZONE	employee_id, full_name, email, job_title, department, salary, building_name, zone_name	HR management — personnel directory across all buildings.
vw_location_audit	USER_LOCATION ⋈ USER_ACCOUNT ⋈ BEACON ⋈ FLOOR ⋈ BUILDING LEFT JOIN ZONE	location_id, full_name, building_name, floor_number, zone_name, beacon_uuid, detected_at	Security review — post-incident user movement analysis.
vw_incident_summary	BUILDING LEFT JOIN INCIDENT GROUP BY building_id	building_id, building_name, total_incidents, active_count, resolved_count, critical_count	Management reporting — risk assessment per building.

Sample View - vw_active_incidents

```
-- Shows all currently active incidents with building context.
-- Used by the admin dashboard to display ongoing emergencies.
CREATE OR ALTER VIEW vw_active_incidents AS
SELECT i.incident_id, b.building_name, b.address,
       i.incident_type, i.severity_level,
       i.start_time, i.description
FROM   INCIDENT i
JOIN   BUILDING b ON i.building_id = b.building_id
WHERE  i.status = 'active';
GO
```

3.5 Indexes

Seventeen non-clustered indexes are created to optimise the most frequently executed queries. Indexes on foreign key columns speed up JOIN operations. Indexes on status and flag columns speed up the WHERE filter conditions used in most system queries.

Index Name	Column(s) Indexed	Justification
idx_floor_building	FLOOR(building_id)	Speeds up loading all floors for a building on layout requests.
idx_zone_floor	ZONE(floor_id)	Speeds up zone lookups when building the navigation map for a floor.
idx_beacon_floor	BEACON(floor_id)	Speeds up beacon-to-floor lookup during user location detection (P3).
idx_beacon_zone	BEACON(zone_id)	Supports optional zone-level beacon filtering.
idx_beacon_status	BEACON(status)	Speeds up active beacon filter on every detection event.
idx_exit_floor	EXIT_POINT(floor_id)	Speeds up exit lookup by floor during BFS route generation (P5).
idx_exit_status	EXIT_POINT(status)	Critical path in BFS — filters available exits on every route request.
idx_exit_emergency	EXIT_POINT(is_emergency_exit)	Speeds up emergency exit lookup queried on every route generation.

idx_incident_building	INCIDENT(building_id)	Speeds up finding all incidents for a building in admin review.
idx_incident_status	INCIDENT(status)	Speeds up active incident query called at every app startup.
idx_route_incident	EVACUATION_ROUTE(incident_id)	Speeds up retrieving all routes for a specific incident.
idx_route_user	EVACUATION_ROUTE(user_id)	Speeds up user evacuation history queries (sp_get_user_history).
idx_step_route	ROUTE_STEP(route_id)	Speeds up fetching all steps for a route — delivered to the user app.
idx_loc_user	USER_LOCATION(user_id)	Speeds up location history queries per user.
idx_loc_time	USER_LOCATION(detected_at)	Speeds up time-range queries on the location audit log.
idx_edge_from	EDGE(from_node)	Critical for BFS graph traversal — finds outgoing edges from a node.
idx_edge_to	EDGE(to_node)	Supports reverse traversal and bidirectional pathfinding.

Sample Index Creation

```
-- Speeds up active incident filter — runs at every app startup
CREATE INDEX idx_incident_status ON INCIDENT(status);

-- Critical for BFS — finds all edges leaving a graph node
CREATE INDEX idx_edge_from ON EDGE(from_node);

-- Emergency exit lookup — runs on every route generation
CREATE INDEX idx_exit_emergency ON EXIT_POINT(is_emergency_exit);
GO
```

3.6 Triggers

Nine triggers are defined using CREATE OR ALTER TRIGGER syntax. They enforce business rules, validate data integrity, and protect audit tables. SQL Server INSTEAD OF triggers are used for validation and blocking — they intercept the operation before it reaches the table, allowing full control over what data gets written.

Trigger Name	Type	Table	Rule	Action & Business Impact
trg_validate_beacon_floor	**INSTEAD OF INSERT**	USER_LOCATION	**BR-12**	Validates that the beacon referenced belongs to the specified floor. If not, raises error 50012 and aborts insert. *Enforces BR-12: a location record can only reference a beacon that belongs to the same floor.*
trg_auto_end_time	**AFTER UPDATE**	INCIDENT	**BR-13**	When status changes from active to resolved and end_time is NULL, automatically sets end_time = GETDATE(). *Prevents resolved incidents from having a NULL end_time — keeps audit data accurate.*
trg_check_end_time	**AFTER UPDATE**	INCIDENT	**BR-13**	Validates end_time > start_time whenever end_time is set. Raises error 50013

				and rolls back if violated. *Enforces BR-13: incident timeline must be chronologically consistent.*
trg_no_duplicate_step	**INSTEAD OF INSERT**	ROUTE_STEP	—	Checks for duplicate (route_id, step_number) before insert. Raises error 50004 and aborts if found. *Backs up the UNIQUE constraint with a descriptive error for debugging.*
trg_default_route_status	**INSTEAD OF INSERT**	EVACUATION_ROUTE	—	Sets route_status = 'generated' if the value is NULL or empty on insert. *Defensive default — prevents NULL or empty route status values.*
trg_no_delete_incident	**INSTEAD OF DELETE**	INCIDENT	**BR-14**	Throws error 50014 on any DELETE attempt on INCIDENT. No records are ever deleted. *Enforces BR-14: incidents are permanent audit records.*

trg_no_delete_route	**INSTEAD OF DELETE**	EVACUATION_ROUTE	**BR-14**	Throws error 50015 on any DELETE on EVACUATION_ROUTE. *Enforces BR-14: routes are permanent historical records.*
trg_no_delete_location	**INSTEAD OF DELETE**	USER_LOCATION	**BR-14**	Throws error 50016 on any DELETE on USER_LOCATION. *Enforces BR-14: location logs are immutable audit data.*
trg_block_exits_on_incident	**AFTER INSERT**	INCIDENT	**P4**	On new active incident: sets status='blocked' on all non-emergency exits in that building. *Automates Process P4: forces occupants to use only designated emergency exits during an incident.*

Sample Trigger - trg_block_exits_on_incident (Process P4)

```
-- After a new active incident is inserted, automatically block
-- all non-emergency exits in that building.
CREATE OR ALTER TRIGGER trg_block_exits_on_incident
ON INCIDENT
AFTER INSERT
AS
BEGIN
   SET NOCOUNT ON;
   UPDATE EXIT_POINT
   SET    status = 'blocked'
   WHERE  is_emergency_exit = 0
   AND    floor_id IN (
      SELECT f.floor_id FROM FLOOR f
      JOIN inserted i ON f.building_id = i.building_id
      WHERE i.status = 'active'
   );
```

3.7 Stored Procedures

Twelve stored procedures are defined using CREATE OR ALTER PROCEDURE syntax with T-SQL parameter declarations (@p_*). Each procedure is linked to a system process from Section 1.6 of the Conceptual Design, making the business logic modular and reusable.

Procedure	Process	Parameters	What It Does
sp_register_building	**P1**	@p_name, @p_address, @p_type, @p_floors	Inserts a new building, returns new building_id via SCOPE_IDENTITY().
sp_add_floor	**P1**	@p_building_id, @p_floor_number, @p_map_ref, @p_height	Adds a floor to a building, returns new floor_id.
sp_register_incident	**P4**	@p_building_id, @p_type, @p_severity, @p_start, @p_desc	Inserts active incident. Trigger fires to block exits automatically.
sp_resolve_incident	**P4**	@p_incident_id	Sets status=resolved, end_time=GETDATE(). Raises error 50004 if not found.
sp_get_available_exits	**P5**	@p_floor_id	Returns available emergency exits for a floor. Input to BFS algorithm.
sp_generate_route	**P5**	@p_incident_id, @p_user_id, @p_start_floor, @p_dest	Creates a route record, returns new route_id.
sp_add_route_step	**P5/P6**	@p_route_id, @p_step_no,	Adds one ordered step to an existing route.

		@p_floor, @p_zone, @p_instr	
sp_log_location	**P3**	@p_user_id, @p_beacon_id, @p_floor_id, @p_zone_id	Logs a beacon detection event with GETDATE() timestamp.
sp_get_user_history	**P7**	@p_user_id	Returns full evacuation history for a user, ordered by date DESC.
sp_get_active_incidents	—	No parameters	Returns all active incidents with building info, ordered by severity.
sp_get_building_stats	—	@p_building_id	Returns floor, incident, employee, beacon, exit counts for one building.
sp_assign_beacon	**P1**	@p_floor_id, @p_zone_id, @p_uuid, @p_major, @p_minor, @p_install	Validates UUID uniqueness (BR-06), then inserts beacon.

Sample Procedure - sp_resolve_incident (Process P4)

```
CREATE OR ALTER PROCEDURE sp_resolve_incident
   @p_incident_id INT
AS
BEGIN
   UPDATE INCIDENT
   SET    status   = 'resolved',
          end_time = GETDATE()
   WHERE  incident_id = @p_incident_id
   AND    status      = 'active';

   IF @@ROWCOUNT = 0
      THROW 50004, 'Incident not found or already resolved.', 1;

   SELECT CONCAT('Incident ', CAST(@p_incident_id AS VARCHAR), ' resolved.') AS
message;
END;
GO
```

3.8 DCL - Access Control

DCL (Data Control Language) creates SQL Server logins, maps them to database users, and grants permissions. Three users are defined matching the three access levels in the non-functional requirements. Role separation is enforced at the database level independently of the application layer.

User / Login	Represents	Permissions Granted	Restrictions
evac_admin	Building managers and safety officers	GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo. GRANT EXECUTE ON SCHEMA::dbo.	None — full access required for system management.
evac_occupant	Occupants using the mobile app	SELECT on BUILDING, FLOOR, ZONE, BEACON, EXIT_POINT, INCIDENT, EVACUATION_ROUTE, ROUTE_STEP. SELECT + INSERT on USER_LOCATION. EXECUTE on all procedures.	Cannot INSERT, UPDATE, or DELETE on INCIDENT, EVACUATION_ROUTE, or BUILDING.
evac_readonly	Auditors and reporting tools	GRANT SELECT ON SCHEMA::dbo.	No INSERT, UPDATE, DELETE, or EXECUTE anywhere.

DCL Statements

```
-- Create SQL Server logins
IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name = 'evac_admin')
   CREATE LOGIN evac_admin WITH PASSWORD = 'AdminPass2024!';
IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name = 'evac_occupant')
   CREATE LOGIN evac_occupant WITH PASSWORD = 'OccupantPass2024!';
IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name = 'evac_readonly')
   CREATE LOGIN evac_readonly WITH PASSWORD = 'ReadPass2024!';
GO

-- Map logins to database users
CREATE USER evac_admin    FOR LOGIN evac_admin;
CREATE USER evac_occupant FOR LOGIN evac_occupant;
CREATE USER evac_readonly FOR LOGIN evac_readonly;
GO

-- Grant permissions
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO evac_admin;
GRANT EXECUTE ON SCHEMA::dbo TO evac_admin;
GO

GRANT SELECT ON dbo.INCIDENT         TO evac_occupant;
GRANT SELECT, INSERT ON dbo.USER_LOCATION TO evac_occupant;
GRANT EXECUTE ON SCHEMA::dbo TO evac_occupant;
GO

GRANT SELECT ON SCHEMA::dbo TO evac_readonly;
GO
```

3.9 Database Deployment Description

The complete database is packaged as a single self-contained T-SQL script file (DB.sql). Executing it in SSMS or via sqlcmd on any SQL Server instance creates the database, all tables, all sample data, and all database objects in the correct dependency order.

The script runs in dependency-safe order:

- Drops and recreates SmartEvacuationDB
- DDL: 15 tables in FK dependency order: BUILDING → FLOOR → ZONE → BEACON → EXIT_POINT → STAIRCASE → STAIRCASE_FLOOR → USER_ACCOUNT → EMPLOYEE → USER_LOCATION → INCIDENT → EVACUATION_ROUTE → ROUTE_STEP → NODE → EDGE
- DML: Data inserted in same dependency order (40+ rows per table)
- DQL: 33 SELECT queries executed (results displayed in SSMS Results pane)
- Views: 7 views created with CREATE OR ALTER VIEW
- Indexes: 17 indexes created with CREATE INDEX + GO
- Triggers: 9 triggers created with CREATE OR ALTER TRIGGER
- Stored Procedures: 12 procedures with CREATE OR ALTER PROCEDURE
- DCL: 3 logins and users created, permissions granted

3.10 Project Conclusions and Results

The Smart Evacuation Navigation System has been fully designed and implemented through three phases - Conceptual, Logical, and Physical - producing a complete, deployable, production-ready SQL Server database that satisfies all project requirements. The database models a real-world B2B safety platform for large multi-story buildings. It stores building structure, BLE beacon hardware, exit safety data, user accounts, employee records, real-time location detection, incident management, and BFS-computed evacuation routes. All 15 business rules from the conceptual design are enforced either by SQL constraints or by database triggers.

Implementation Summary

Database System:	Microsoft SQL Server | Language: T-SQL
Script File:	DB.sql
Total Tables:	15 tables, all normalised to 3NF
Records:	40+ rows in every table
DQL Queries:	33 queries (Q01–Q33) aligned with Section 2.6 Relational Algebra
Views:	7 views (CREATE OR ALTER VIEW)
Indexes:	17 non-clustered indexes
Triggers:	9 triggers (CREATE OR ALTER TRIGGER)
Stored Procedures:	12 procedures (CREATE OR ALTER PROCEDURE)
DCL Users:	3 logins and database users with role-based permissions